DATA STRUCTURE LAB

Array [illegible] ArrayList

// [illegible]

length

[illegible]

size ≠ capacity

[illegible] ≠ [illegible]

□□□□ API

```
int[] scores = {85, 92, 78};
scores[1] = 95;
int n = scores.length;
```

□□□□ [] · □□□□

```
List<Integer> scores = new ArrayList<>();
scores.add(85);
scores.set(0, 95);
int n = scores.size();
```

□□ API · □□□□□

size [illegible] capacity [illegible]

10	20	30	[illegible]

size = 3

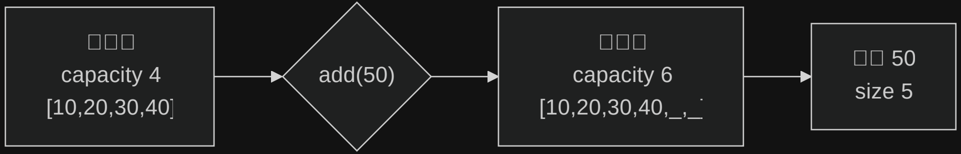
[illegible]

capacity = 4

[illegible]

capacity [illegible] List API [illegible]

[illegible]



4 → 6

4 + floor(4 / 2)

[illegible] 4 [illegible]

[illegible] add [illegible] O(n)

[illegible] O(1)

[illegible]

OpenJDK [illegible] 1.5×[illegible] List [illegible]

```
List<Order> orders = new ArrayList<>(incoming);
```

[illegible]

A	B	C →	D →	E →

[illegible] add

[illegible]

index 1 [illegible]

[illegible]O(n)

Array length | ArrayList size / capacity

0	1	2	3
10	20	30	

size = 3 · capacity = 4 · 0 · 0

40 add 1 insert

capacity add insert remove

primitive □ wrapper

```java
int[] raw = {1, 2, 3};
// □□□ int □
```

Array □□□□ primitive□ `int` □ `double` □ `char`

```java
List<Integer> boxed = new ArrayList<>();
boxed.add(1);           // int → Integer
int n = boxed.get(0);   // Integer → int
```

□□□□□□ `List<int>` □□□ wrapper □ autoboxing□

□□□□□□□

```java
List<Integer> values = new ArrayList<>();
values.add(null);

Integer boxed = values.get(0); // □□□boxed == null
int raw = values.get(0);       // NullPointerException

Integer maybe = values.get(0);
if (maybe != null) {
    int safe = maybe;
    System.out.println(safe);
}
```

□□□□ get() □□□□□□□ `Integer → int` □□□□□□□

Arrays.asList □□□□□□□

```java
List<String> readOnly = List.of("Ada", "Linus");
// set / add □□□□
```

`Arrays.asList` □□□□□□□□ view□□□□□ ArrayList□

□□□□□□□□□□

```java
import java.util.*;

class Roster {
  public static void main(String[] args) {
    List<String> names =
        new ArrayList<>(List.of("Ada", "Linus"));
    names.add("Grace");
    System.out.println(names);
  }
}
```

□□

1. □□□□□□ String[]
2. □□□□□□□□□□□□
3. □ Monaco □□□□□□□ Java

[illegible]

[illegible]	Array	ArrayList
`get(index)`	O(1)	O(1)
`set(index)`	O(1)	O(1)
[illegible] `add`	[illegible]	[illegible] O(1)
[illegible]/[illegible]	[illegible] O(n)	O(n)
[illegible]	[illegible]	[illegible] O(n)

Big-O [illegible]boxing[illegible]

[illegible]

[illegible]

[illegible] 7 [illegible]primitive [illegible]

`double[]`

[illegible]

[illegible]

`ArrayList`

[illegible] 8 × 8

[illegible]

`char[][]`

[illegible]

[illegible]

`ArrayList`

[illegible] primitive [illegible] Array[illegible] ArrayList[illegible]

□□□□□

size □□□□□capacity □□□□□□

□□□ 1.5×□□□□□□□□□□□□□□□

primitive □□□□□□□□□ API □□□

□□

□□□□LinkedList□Set□Map □□□□□

choose by behavior, measure by evidence